

我的 AI Agent 使用手冊：把工具清單變成工作流

Sheng-Yuan Chen

2026-06-11

1 前言：為什麼要寫一本「使用手冊」

用 Claude Code 一段時間後，我累積了一大堆 skills——研究、寫作、投影片、數據分析、發筆記，零零總總幾十個。問題是：工具一多，反而不知道什麼時候該用哪個。更常見的狀況是，同一個 session 裡東問西問，最後 AI 開始「忘東忘西」、品質下滑。

於是我把自己的用法整理成一本手冊，回答三個問題：(1) 我有哪些工具、什麼情況用哪個？(2) 怎麼把零散的工具串成「工作流」，而不是每次重想？(3) 怎麼有效率地用 AI agent——省 token、管理 context、寫好 CLAUDE.md、什麼任務該開哪家 AI？這篇是那本手冊的公開版，內容裡所有關於 Claude Code 的事實都對照了 Anthropic 官方文件。

2 先建立正確的心智模型

2.1 AI agent ≠ 聊天機器人

聊天機器人是「你問、它答、它等你」。Agent (像 Claude Code) 是「你描述目標，它自己讀檔、跑指令、改檔、驗證、迭代」——你在旁邊看、必要時導正，或直接走開。對研究的意義是：別再「自己想好每一步、一句句下指令」，而是把目標、限制、驗收標準講清楚，讓它自己跑。比方說，不要說「幫我把這個變數 demean」，而是說：「用 first-difference 估這個 panel DML，跑完給我 bias / SE 表，並和 OLS 對照。」

2.2 三個延伸機制：Skills / CLAUDE.md / Memory

機制	是什麼	角色
Skills	一包 SKILL.md + 腳本，給 AI 特定領域知識與可重複流程	只在相關時才載入的「專門流程」
CLAUDE.md	每次開場自動載入的「專案說明書」	每次都載入的「常駐規則」
Memory	跨 session 的長期記憶（偏好、專案狀態）	你的「個人筆記」

一句話記住分工：CLAUDE.md = 每次都載入的常駐說明；Skill = 需要時才載入的專門流程；Memory = 你的偏好與狀態。三者別混用——這在省 token 的部分很關鍵。

3 把技能整理成「工作流」，而不是「工具清單」

工具清單會讓你每次都重想；工作流則是把「某個常見任務」固定成一條串接。以下是幾個對研究生最實用的例子（觸發句直接貼給 agent 即可）。

- **讀懂並報告一篇論文**：讀 PDF → 建立「理解 → 大綱 → 逐張 frame → 版面」的中介表示 → 寫 .tex → 逐頁視覺檢查。別讓 AI 從 PDF 直接跳到投影片，中間要有可檢查、可修補的階段。
- **多篇文獻找研究缺口**：做跨論文 synthesis（不要逐篇摘要），把 gap 分類並評 Strong / Medium / Weak，每個 claim 標出論文出處，不確定的標 to verify。
- **從想法到論文初稿**：先讓 AI 用問答把想法逼成一份 SPEC.md（它會問你沒想到的取舍與 edge case），再開新 session 執行——新 session context 乾淨、又有 spec 可循。
- **數據分析到出表 / 圖**：要求可重現的腳本 + 證據；data/raw/ 不要動，清理後存 data/processed/。

共通原則：給它一個能跑的「驗收檢查」。最有效的一招是給 AI 一個能自己跑、會回傳 pass / fail 的檢查（測試、編譯、跑回歸對照、screenshot 比對）。有了它，AI 會自己跑到通過，你不用當「人肉驗證迴圈」；並且要求它給證據，而不是一句「我做好了」。

4 高效使用 AI agent 的核心：context 是最稀缺的資源

4.1 為什麼 AI 用久了會「變笨」：context rot

AI 的 context window（脈絡視窗）裝著整段對話——每則訊息、每個被讀的檔、每次指令輸出。它會很快填滿，而且越滿、表現越差：模型開始「忘記」前面的指令、變多錯。Anthropic 把這現象叫 context rot：transformer 要處理 n^2 的 token 兩兩關係，context 越長、注意力被攤越薄。結果是準確率隨 context 變長而漸降（是斜坡、不是斷崖）。核心心法一句話：用「最小的高訊號 token 集合」達成任務，不是塞越多越好。

4.2 省 token 的具體做法

1. **任務之間 /clear**。換到不相關的工作就清空 context；最常見的浪費就是「kitchen sink session」。
2. **同一問題被你糾正超過兩次** → /clear 重開，用更好的 prompt。乾淨 session + 好 prompt 幾乎總是贏過拖很長的 session。
3. **階段做完就 /compact**（不是等到變笨才壓），可加指示如 /compact 著重在 R 程式。
4. **用 /context 看誰在吃 context**。
5. **要 AI 去讀檔，而不是貼長文進來**。用 @ 檔名引用、或叫它自己讀檔 / 抓網址。
6. **大量讀檔的探索丟給 subagent**——它在獨立 context 探索，只回報約 1,000–2,000 token 的摘要。

- 7. 小問題用 side question，答案不進對話歷史。
- 8. CLAUDE.md 保持精簡——它每次都載入、整段常駐。

4.3 CLAUDE.md 是什麼、怎麼寫

定義：每次對話開場 AI 都會自動讀的「專案說明書」。放它從程式碼推不出來、但每次都需要的东西：常用指令、程式風格、工作流規則、專案慣例。位置會自動合併：

位置	作用
~/.claude/CLAUDE.md	全域，套用到所有 session
./CLAUDE.md（專案根）	專案層，commit 進 git 與人共享
./CLAUDE.local.md	個人專案筆記，記得加進 .gitignore
父 / 子目錄	monorepo 自動往上拉、讀子目錄檔時才拉子目錄的

官方寫法守則：保持短、人類可讀，每一行問「拿掉這行會讓 AI 犯錯嗎？」不會就刪；臃腫的 CLAUDE.md 會讓 AI 忽略你真正的指令。該放：猜不到的指令、與預設不同的風格、測試方式、架構決定、環境怪癖。不該放：讀程式就知道的事、會頻繁變動的資訊、長篇教學。加 IMPORTANT / YOU MUST 可提高遵從度；在對話裡按 # 可把一句話直接寫進 CLAUDE.md。我自己把專案根的 CLAUDE.md 當成路由表，只寫「哪個任務先讀哪個 STATUS.md」，把會變動的進度交給各專案的 STATUS.md。

4.4 先探索 → 再計畫 → 才動手

官方建議四階段：Explore（plan mode 只讀不改）→ Plan → Implement → Commit。但 plan mode 有 overhead：小事直接做，只在「不確定做法、會動多個檔、對該段不熟」時才用。一句話能描述改動，就跳過計畫。

5 不同 AI 的優勢與選用

模型版本與 benchmark 變很快，下面是 2026 年中的「相對強項」歸納，確切數字請臨用前再查。

模型家族	相對強項	什麼情況開它
Claude（Anthropic）	程式 / agentic 任務、複雜邏輯與除錯、長流程、最不「AI 腔」的英文寫作	agentic 工作流、寫改 R / LaTeX、用自己口吻寫英文論文
GPT（OpenAI）	通用、知識面廣、結構化推理、外掛生態	快速廣博的解法、brainstorming、工具整合
Gemini（Google）	速度快、context window 超大、原生 Google 搜尋、多模態	一次塞超大量文字、即時網路資訊、處理圖片影片

實務建議：agentic 流程 + 寫 code + 英文論文寫作走 Claude / Claude Code；超長一次性閱讀可考慮 Gemini 的大 context；快速查證用帶搜尋的模型。不要迷信單一 benchmark 冠軍——挑模型看任務類型。Claude 家族內部：Opus 做最難的推理與長流程、Sonnet 是又快又強的日常主力、Haiku 處理量大重複的小事。

6 小結

主題	一句話心法
心智模型	描述目標與驗收標準，讓 agent 自己跑，別當它的手腳
workflow	把常見任務固定成串接，附「觸發句」，每次複製即用
驗收	給一個能自己跑、回傳 pass/fail 的檢查；要證據不要「我做好了」
context	它是最稀缺資源；越滿越笨（context rot）
省 token	任務間 /clear、階段後 /compact、讀檔別貼長文、探索丟 subagent
CLAUDE.md	短、只放猜不到又每次要的；會變動的進度交給 STATUS.md
選 AI	看任務類型選家族；Claude 做 agentic / 寫作、Gemini 大 context、GPT 廣博

工具會一直變多，但「先理解、再規劃、才動手；管好 context；給它能驗證的檢查」這幾條原則不太會過期。

參考來源：Anthropic, *Best practices for Claude Code*；*Manage Claude's memory*；*Effective context engineering for AI agents*。模型比較為第三方 2026 年資料，數字會過期，僅供方向參考。